

增加 ROI 的优化方案：

当前全图扫描的方式虽抗干扰强，但每一帧都对整幅图像进行二值化、轮廓提取、装甲板筛选和 PnP 解算，会造成比较高的 CPU 占用。尤其在高分辨率输入下，背景灯条、地面反光、裁判系统灯条和队友装甲板都会进入候选区域，最后导致很多无效轮廓参与计算。

比较合适的优化方向是引入 ROI 动态跟踪机制。某一帧成功识别到目标装甲板后，记录该装甲板的中心点、宽高、类别、时间戳和滤波器预测状态。下一帧先根据上一帧目标状态预测它在图像平面上的位置，只在预测中心附近开一个局部 ROI 做识别。这样可以减少图像处理面积，也能降低背景杂光进入候选集的概率。

这个方案有一个问题：如果画面其他区域突然出现优先级更高的装甲板，例如英雄、哨兵、残血目标或者更近目标，系统仍然盯着旧目标附近，就可能错过更优目标。所以 ROI 不应该完全替代全局搜索，更合理的是 ROI 检测为主、周期性全局检测为辅。比如每隔一两帧执行一次全局检测，或者在 ROI 连续失败、云台快速转动、目标优先级需要重新判断时强制回到全图搜索。

实践的时候需要把全图检测和 ROI 检测分开记录。比较有用的量化项包括平均耗时、P95/P99 耗时、CPU 占用率、候选轮廓数量、ROI 面积占比、全图回退次数、目标切换延迟和漏检率。这样才能判断 ROI 到底是在提高系统效率，还是只是让系统更容易锁死在旧目标上。

EKF 增加维度或者更换成 UKF 的方案：

目前的 EKF 状态向量如果只考虑匀速运动，那么目标一旦出现明显加速度，预测点就会继续沿旧速度方向外推，瞄准点可能偏到装甲板侧边甚至直接脱靶。比较直接的改法是在状态向量里加入加速度统计。如果还要描述目标绕车体中心旋转的情况，可以进一步考虑角度、角速度和角加速度。不过加速度模型并不一定能解决所有问题。当机器人高速旋转时，装甲板的位置变化并不是简单的线性变化，而是非线性运动。EKF 依靠局部线性化近似，在旋转较快或者速度变化很剧烈时，可能会出现较大的预测偏差，表现出来就是弹道打在装甲板侧面。

UKF 的好处是它不用像 EKF 那样只依赖切线近似, 对高速旋转、坐标系频繁转换这类非线性场景会更友好。自瞄里经常需要在相机坐标系、云台坐标系和世界坐标系之间转换, 有时还会在极坐标和直角坐标之间切换, UKF 在理论上能更稳地处理这种非线性变化。

但是 UKF 它会引入更多非线性计算, 可能让滤波部分变慢。实际是否值得换, 不能只看视频效果或者单次命中情况, 而要用同一批回放数据对比 EKF、加入加速度的 EKF 和 UKF。建议重点记录预测点像素误差、yaw/pitch 预测误差、急停急加速场景下的收敛时间、小陀螺命中窗口捕获率、滤波单帧耗时、总延迟变化和 CPU 占用变化。

github 上有现成的针对齐苗的优化:

https://github.com/wudideshayushayu/UKF-for-sp_vision_25

EKF 和 UKF 的对比视频:

http://bilibili.com/video/BV1zi42117XM/?spm_id_from=333.337.search-card.all.click&vd_source=670fc4d558f034d9cc2b1d153e59cc80

6 阶卡尔曼和加速度统计资料(资料要钱,但是首页的 AI 简介里面提到了 6 阶的有点):

<https://wenku.csdn.net/doc/4o15kuto1u?>

📄

基于六阶卡尔曼滤波的RMUC自瞄预测系统实现

版权申诉

📄

限时特惠: ¥29.90

ZIP文件

50.56MB | 更新于2024-09-30 | 90 浏览量 | 8 评论 | 举报

1 ⭐ 收藏

AI解读

卡尔曼滤波 (Kalman Filter, KF) 是一种递归式最优估计算法, 广泛应用于导航、制导、目标跟踪、传感器融合、机器人定位及智能车自瞄系统等动态系统状态估计场景中。本项目标题明确指出“基于六阶卡尔曼滤波实现的RMUC自瞄预测C源码+文档说明”, 其技术内核远超常规四阶或五阶设计, 具有显著的工程深度与理论严谨性。需特别强调: 所谓“六阶”并非指滤波器阶数在数学意义上的微分方程阶次, 而是指系统状态向量维度为6——即状态空间模型中包含六个独立且物理可解释的状态变量, 典型构成为: 目标在图像坐标系 (或世界坐标系) 下的横坐标x、纵坐标y、横向速度vx、纵向速度vy、横向加速度ax、纵向加速度ay。该六维状态向量完整刻画了目标在二维平面内的运动学全貌, 支持对非匀速、带加速度扰动的目标轨迹进行高精度建模与前瞻预测, 是RMUC (RoboMaster University Competition, 全国大学生机器人大赛RoboMaster机甲大师赛) 中全自动步兵机器人实现“自瞄—识别—预测—开火”闭环控制的关键算法基石。

RMUC赛事对视觉伺服系统的实时性、鲁棒性与预测精度提出极端严苛要求: 摄像头帧率通常为30–60Hz, 而弹丸飞行时间约为120–200ms, 若仅依赖当前帧检测结果直接瞄准, 将因系统延迟导致严重脱靶。因此必须构建具备时间外推能力的运动预测模型。六阶卡尔曼滤波在此处发挥核心作用: 它以离散时间线性/近似线性状态空间模型为前提, 融合IMU数据 (若有)、视觉检测结果 (如OpenMV或ZYNQ/FPGA图像处理模块输出的目标中心像素坐标) 作为观测输入, 通过预测步 (Predict Step) 与更新步 (Update Step) 交替迭代, 在每一帧周期内完成对目标未来N帧 (如T=150ms对应3–5帧) 位置的最优估计。其状态转移矩阵F精确编码了运动学先验知识——例如采用“恒定加速度模型” (Constant Acceleration Model, CAM), 则F矩阵由泰勒展开推导得出, 显式包含 Δt 、 $\Delta t^2/2$ 等时间尺度项; 过程噪声协方差Q则量化了加速度突变、机械振动、图像检测抖动等不确定性; 观测矩阵H将六维状态映射至二维图像坐标 (x,y), 并可通过引入镜头畸变校正参数或透视投影矩阵扩展为非线性形式 (此时需切换至EKF或UKF); 观测噪声R则表征视觉识别误差、光照变化干扰等测量不确定性。整个滤波流程完全在嵌入式端实时运行, 代码严格遵循C99标准, 无动态内存分配、无浮点库依赖 (采用定点数或CMSIS-DSP优化浮点运算), 适配ARM Cortex-M系列MCU (如STM32H7系列), 在Keil MDK-ARM v5.x环境下完成编译链接, 资源占用可控 (RAM < 8KB, Flash < 64KB), 中断响应延迟低于50 μ s, 满足RMUC硬实时约束。

项目文档不仅涵盖算法原理推导 (含状态方程、观测方程、卡尔曼增益矩阵K的递推公式、协方差传播律P和P'的更新逻辑), 更深入剖析了六阶设计相较传统二阶 (仅x,y)、四阶 (x,y,vx,vy) 模型的本质优势: 四阶模型假设目标匀速运动, 在目标急停、转向、跳跃时预测发散严重; 而六阶模型通过显式建模加速度状态, 赋予系统对机动目标的强适应能力——当检测到连续几帧速度变化率显著增大时, 滤波器能自动调整协方差矩阵P, 提升加速度状态的不确定性权重, 从而抑制过拟合, 增强鲁棒性。源码中kalman_test-master目录结构清晰, 包含kalman_filter.c/h (核心滤波器实现)、motion_model.c/h (状态转移与观测映射)、target_predict.c/h (预测接口封装)、main.c (主循环调度与传感器数据同步)、config.h (滤波器参数可调宏定义, 如Q/R初值、采样周期 Δt 、预测步长N)。所有函数均添加详尽Doxygen注释, 关键路径插入CMSIS-DSP加速指令 (如arm_mat_mult_f32), 矩阵运算避免朴素三重循环, 采用分块计算与寄存器缓存优化。测试验证部分包含MATLAB离线仿真比对 (生成真值轨迹与滤波输出曲线)、STM32CubeMonitor实时数据流监控、以及RMUC实战录像回放验证——在10m距离、目标横向移动速度2.5m/s、加速度峰值 $\pm 1.8\text{m/s}^2$ 的极限工况下, 预测误差稳定控制在 ± 8 像素 (对应实际偏差 $\leq 1.2\text{cm}$), 命中率提升至91.7%, 远超赛事平均水平。该毕设项目之所以获评96分高分, 正在于其将抽象控制理论、嵌入式软硬件协同、机器人运动学、计算机视觉与竞技工程实践深度融合, 形成一套可复现、可移植、可扩展的自主瞄准技术范式, 为后续引入多模型交互滤波 (IMM)、深度学习辅助观测 (如YOLOv5轻量化检测+KF轨迹优化)、或迁移到ROS2机器人平台提供坚实底层支撑。

收起

延迟量化方案：

FPS 只能说明程序每秒处理多少帧，不能说明当前输出的控制量对应的是多久以前的画面。自瞄真正关心的是图像年龄，也就是一帧图像从摄像头采集，到识别、滤波、发送、执行，再到弹丸飞到目标位置之间经过了多久。

首先需要给每一帧图像绑定时间戳。理想情况下，这个时间戳应接近摄像头曝光完成的时刻；如果无法获取硬件时间戳，就在程序成功取到图像的瞬间记录时间。后续识别、滤波和弹道预测都必须沿用这个时间戳，不能在识别线程里重新生成，否则会把图像在队列中等待的时间掩盖掉。

一帧图像进入系统后，建议记录图像采集时间、开始识别时间、识别结束时间、滤波结束时间、串口发送时间、下位机接收时间、云台响应时间和发射时间。这样可以把总延迟拆成视觉识别延迟、滤波预测延迟、串口通信延迟、电控处理延迟、云台机械响应延迟和弹丸

飞行延迟。

视觉识别延迟主要来自颜色空间转换、二值化、形态学处理、轮廓提取、装甲板筛选、数字识别和 PnP 解算。如果这一部分耗时较高，可以通过 ROI 区域跟踪、减少全图扫描、减少无效轮廓、关闭调试图像保存、减少图像拷贝，以及使用 NPU 或 RGA 加速来优化。

滤波器更新时必须使用图像采集时刻，而不能使用处理完成时刻。因为识别出的装甲板位置对应的是过去某一帧画面，如果把它当作当前状态更新，会造成系统性滞后。预测时应把目标状态外推到弹丸命中时刻。

总延迟可以近似理解为图像年龄、通信与电控延迟、云台响应延迟和弹丸飞行时间的总和。例如图像采集到串口发送用时 18 ms，通信和电控处理 4 ms，云台响应等效延迟 15 ms，目标距离 5 m，弹速 25 m/s，则弹丸飞行时间约为 200 ms，总预测时间约为 237 ms。也就是说，系统应该预测 237 ms 后目标的位置，而不是只预测当前画面中的位置。

最后，延迟不能只看平均值，还要看最大延迟和 P95/P99 延迟。平均延迟低但波动大，仍然会导致打击不稳定。对自瞄系统来说，稳定的延迟更容易补偿，随机抖动的延迟反而更难处理。

浙江师范大学的方案：预测时间 = 飞行时间 + 串口通讯延迟 + 发弹延迟 + 程序延迟。https://github.com/dielivelr/Z_LION_AutoAim2025?

评估自瞄效果时，不能只用单一命中率概括。一个系统可能静态靶命中率很高，但遇到无规则机动、目标切换或者小陀螺就明显变差。比较合理的做法是把测试场景拆开，让每个问题都有对应指标。

无规则机动主要用来观察滤波器面对急停、急加速和突然变向时的表现。目标切换主要用来观察系统是否会被旧目标拖住，尤其在引入 ROI 后，这一项很重要。滤波器收敛速度则适合衡量目标首次出现、遮挡后重捕获、切换目标后的稳定时间。命中率仍然要记录，但最好按距离、速度、小陀螺状态和目标类别分桶统计，而不是只给一个总数。

这部分可以参考这篇评估方案，它提到了无规则机动、目标切换、滤波器收敛速度和命中率等评价方式：<https://zhuanlan.zhihu.com/p/416449365?>

天津大学的思路也值得重点看：把取流、预处理、推理、后处理、串口发送拆成流水线，

并记录每段队列长度和等待时间。这样做的意义不只是提高 FPS，更重要的是能看见每一帧到底卡在哪一段，以及队列是否正在堆积。

如果只记录模块耗时，很容易漏掉队列等待的问题。比如某一帧识别本身只花了 8 ms，但它在识别队列前面等了 20 ms，那么最终输出的控制量仍然对应一张更旧的图像。对于自瞄来说，这种图像年龄比单帧耗时更关键。

性能压榨可以和前面的 ROI 结合起来看。ROI 主要减少传统视觉处理面积和无效轮廓，流水线主要提高整体吞吐并减少模块互相阻塞，推理加速则降低单帧瓶颈。最后需要一起观察 CPU/GPU 占用、温度、功耗、P95/P99 延迟、队列长度和长时间运行是否降频。链接：
<https://github.com/HHgzs/OpenRM-2024>

总结

目标	相关资料	主要看
压榨小电脑性能	HHgzs/OpenRM-2024; ROI 区域跟踪思路	流水线拆分、队列等待、处理面积减少、CPU/GPU 占用和 P95/P99 延迟。
降低传统视觉耗时	ROI 区域跟踪思路	全图扫描和局部识别的耗时差异、候选轮廓数量、回退次数和目标切换延迟。
优化加速或急停预测	6 阶卡尔曼资料; UKF-for-sp_vision_25	加速度统计、预测点误差、滤波收敛速度和滤波本身的耗时增加。
优化小陀螺或高速旋转	UKF-for-sp_vision_25; UKF 对比视频	非线性旋转轨迹、命中窗口捕获率、yaw/pitch 预测误差和 UKF 的算力代价。
量化总延迟	Z_LION_AutoAim2025; 延迟量化方案	图像年龄、程序延迟、串口延迟、发弹延迟、飞行时间和总预测时间。
评价综合自瞄效果	知乎评估方案	无规则机动、目标切换、滤

		波器收敛速度和分工况命中率。
--	--	----------------